```
// Helpers: parseFKLink, collectSubtree, findAnchor, buildCountCTE.
// Pattern: AGG[COUNT(*), no GROUP BY] over a >=3-table star join.
bool JoinDecomposeToPreaggregatedCount::Matches(
    const LogicalOperator &op) const {
  if (op.type() != LogicalOperatorType::AGGREGATE) return false;
  const auto &agg = op.Cast<AggregateOperator>();
  if (!agg.group_by().empty() || !hasCountStar(agg)) return false;
  std::vector<ScanMeta> scans;
  std::vector<std::string> preds;
  collectSubtree(*op.children()[0], scans, preds);
  if (scans.size() < 3) return false;
  std::vector<FKLink> fks;   // equality preds only
  for (const auto &p : preds)
    if (auto f = parseFKLink(p)) fks.push_back(*f);
  if (fks.size() < 2) return false;
  const std::string anchor = findAnchor(fks);
  if (anchor.empty()) return false;
  int non_anchor = 0;
  for (const auto &s : scans) {
    if (s.alias == anchor) continue; ++non_anchor;
    bool ok = false;
    for (const auto &f : fks)
      if ((f.la==s.alias && f.ra==anchor)
          ||(f.ra==s.alias && f.la==anchor)){ ok=true; break; }
    if (!ok) return false;
  }
  return non_anchor >= 2; }
```